

POLITECNICO DI TORINO



Master Degree in Mechatronic Engineering
2025/2026 Academic Year

Robotics

Deep Reinforcement Learning applied to the Collision Avoidance problem

Group 21

Benincasa Mattia - 360167

Vitullo Nicolas - 361762

Quaranta Valeria - 360276

Monte Alessandro - 360583

13 June 2026

1 Introduction

In this report, the application of the Deep Reinforcement Learning to the problem of Collision Avoidance is investigated. In particular, the main goal of this work is to replicate the main results of [1] using a DDQN agent with different values of the epsilon decay rate parameter β . This approach allows to compare different training strategies where a different weight is given respectively to the exploration and exploitation phases of the training stage. This technique is compared with a more classical approach like the DWA algorithm. Then, in addition to the problem of collision avoidance, also the problem of reaching a target position using only a DRL agent is analyzed. A possible solution to this problem is proposed.

1.1 Use of LLMs and contribution of each member

Large Language Models (LLMs) were employed as a supporting tool during the software development process. In particular, they were used to accelerate the implementation of utility functions related to simulation management, such as robot respawning procedures and parsing of Gazebo world files, as well as to provide an initial implementation of the DDQN agent architecture based on TensorFlow/Keras. Moreover, the LLMs were used to provide a support in understanding the main concepts related to DRL topic. All generated code was subsequently reviewed, adapted, tested, and integrated into the software architecture reported in Figure 1.

The design decisions concerning the software architecture, training strategy, hyperparameter tuning, and overall project structure were jointly discussed and agreed upon by all group members. Once the main design choices had been established, the implementation activities were distributed among the team members. In particular, Mattia Benincasa was responsible for the creation of the Gazebo environments and for modifications to the software architecture; Alessandro Monte implemented the robot spawning and reset mechanisms required for training and testing procedures; Nicolas Vitullo developed and integrated the Dynamic Window Approach (DWA) controller; and Valeria Quaranta designed the robot model and implemented the management of robot state acquisition and command generation.

2 Software architecture

To develop the ROS software, the layered architecture described in [1] is adopted and further modified by substituting the Actuation Layer with a dedicated Simulation Layer. The overall architecture is shown in Figure 1 where each package is responsible for a specific functionality. This, guarantees modularity and allows an easy extension on real hardware.

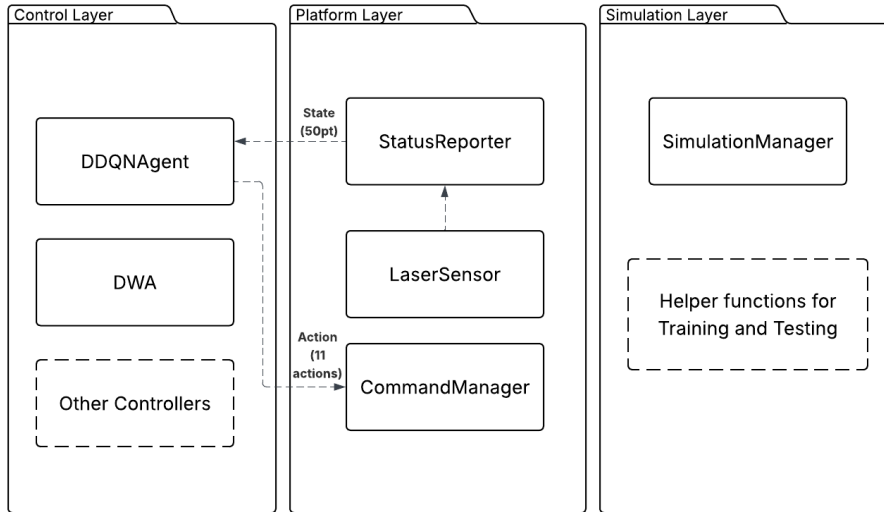


Figure 1: High level scheme of the Software Architecture. The **Control Layer** contains the control algorithms and the Deep Reinforcement Learning agent responsible for obstacle avoidance. The **Platform Layer** is able to translate raw data states into a readable format for the controller. It is also responsible for actuating the correct command it receives from the control layer. **Simulation Layer** includes all the components related to the management of the simulation environment and the training and testing process (respawn of the robot, management of the training and testing maps and so on).

3 Classic approach: DWA

The Dynamic Window Approach (DWA) is a classical local planner that samples velocity pairs (v, ω) within the robot’s dynamic window (bounded by acceleration limits), forward-simulates each trajectory, and selects the one with the lowest combined cost (obstacle proximity, goal alignment, and speed).

This algorithm was tested following the setup of the reference paper, utilizing the exact same environment (Map 1). However, since the original authors do not specify which global planner was coupled with their DWA implementation, a manual waypoint-based approach was adopted for this study. Because DWA is a purely reactive local planner with no memory of the environment, some form of global guidance is strictly necessary to navigate complex paths.

Consequently, ten waypoints were manually defined to safely route the robot from the spawn area to the target zone through the map’s narrow passages. The obtained test results, including the generated trajectory and the specific positions of the waypoints, are illustrated in Fig. 2.

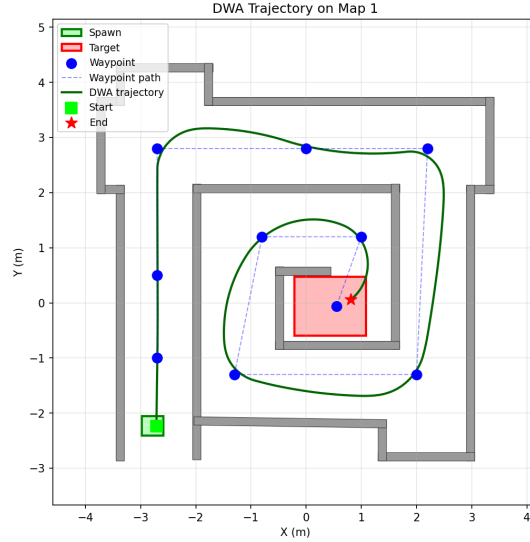


Figure 2: DWA trajectory on Map 1.

4 DRL approach

4.1 DDQN agent setup

The agent is trained using the Double Deep Q-Network (DDQN) algorithm [1]. To mitigate the overestimation bias inherent in standard DQN, the architecture decouples action selection from action evaluation using two separate neural networks: an *online* network and a *target* network.

The state and action spaces replicate the reference paper: 50 LIDAR range readings are mapped to 11 discrete angular velocities between -0.8 and $+0.8$ rad/s. Regarding the network architecture, the baseline setup proposed by [1] is maintained. It consists of two hidden layers with 300 neurons each and ReLU activations, followed by a linear output layer for the Q-values. The reward function is strictly identical to the original framework, utilizing a simplistic yet highly effective step-based structure:

$$R(s, a) = \begin{cases} -1000 & \text{if a collision occurs,} \\ +5 & \text{otherwise.} \end{cases}$$

This specific formulation is intentionally designed to prioritize local safety over global, goal-oriented navigation. The severe penalty of -1000 acts as a critical deterrent, ensuring that the neural network heavily penalizes any state-action pairs that result in a collision, thereby prioritizing the physical integrity of the robot. Conversely, the constant $+5$ reward functions as a continuous survival incentive, encouraging the agent to prolong the episode by actively seeking safe, obstacle-free trajectories. By deliberately omitting a dense, goal-directed reward component (such as minimizing the distance to a target), the agent focuses exclusively on the reactive collision avoidance task. This allows the network to learn a robust local maneuvering policy that is completely independent of the global path.

For the ϵ -greedy exploration strategy, the exploration rate ϵ decays exponentially from an initial value of 1.0 down to a minimum of 0.05. To rigorously evaluate the balance between exploration and exploitation, we tested the exact same set of exponential decay rates ($\beta \in \{0.995, 0.997, 0.999\}$) investigated by the original authors.

Since the reference paper does not explicitly disclose the complete set of training hyperparameters, several key variables were established through empirical tuning. Specifically, the discount factor was set to $\gamma = 0.99$; this

relatively high value ensures the agent heavily weights long-term rewards, which is essential for navigating extended environments. Furthermore, the experience replay mechanism was carefully tuned: the buffer was configured with a capacity of 30,000 transitions—large enough to ensure a diverse, uncorrelated distribution of past experiences, yet bounded to preserve memory efficiency. From this memory, a mini-batch of 64 is randomly sampled per update, achieving an optimal balance between computational speed and gradient stability.

5 Simulation and obtained results

The training and the testing have been conducted on the same maps of the reference paper.

Both testing and training are started through launch files, which handle the configuration of the virtual world, load the robot’s URDF model, and spawn it into the simulation.

During the training, the robot collided most of the times, but reward trend was overall increasing, as can be seen in Figure 3. Of course, the number of steps per episodes increased accordingly. Moreover, it is possible to remark the fact that, for the highest value of β the convergence to a stable reward is slower than the other two values since a higher value of β caused the robot to explore more actions at first. So, the relation between β and exploration/exploitation, described in the paper, has been verified.

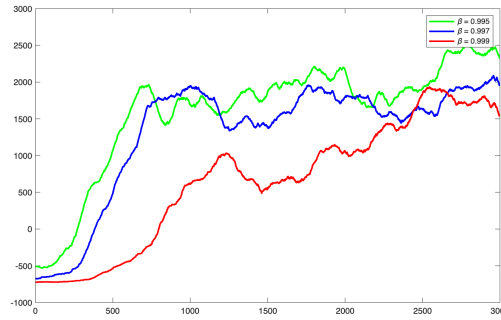


Figure 3: Comparison between the different rewards obtained in simulation processed by using a moving average filter with a window size of 500.

Each test lasts about 300 seconds, which correspond to about 2900-3000 steps. Results varied for different decay rate. Simulation outcomes are reported in 1.

Decay rate	Total steps	Collisions
0.995	2984	1
0.997	2878	24
0.999	2988	0

Table 1: Simulation results

5.1 DRL for target reaching: a possible implementation

Starting from the DRL agent implemented for collision avoidance, a step further is to setup an agent that is able to reach a target while avoiding collisions. First of all, the state space has been modified to include the relative distance and orientation of the robot with respect to the target. Since the task is more complex, a curriculum learning approach has been adopted, starting from a simple scenario without obstacles and gradually increasing the distance between the robot and the target, as well as introducing obstacles in the environment. The reward function has been designed to encourage the agent to reach the target while avoiding collisions, in particular by giving a positive reward when moving towards the target. Due to time constraints it hasn’t been possible to obtain significant results, but the preliminary tests shows that the agent is able to learn to reach the target in the first three scenarios of the curriculum.

6 Conclusion

DRL is the right approach for collision avoidance when the environment is unknown, since it allows the agent to learn a policy that maps states to actions without requiring a model of the environment. However, it has some disadvantages, such as the difficulty in tuning the reward function when dealing with multi-objective tasks (e.g., reaching a target while avoiding collisions) and the long training time required to achieve good performance.

References

- [1] S. Feng, B. Sebastian, and P. Ben-Tzvi, “A collision avoidance method based on deep reinforcement learning,” *Robotics*, vol. 10, no. 2, p. 73, 2021.